

PRESIDENCY UNIVERSITY

CSE211: Data Structures and Algorithms

Lecture 03: Linear Data Structures

Abstract List ADT, Arrays, Dynamic Arrays, and Linked Lists

Afzal Hossan

Lecturer

Department of Computer Science and Engineering

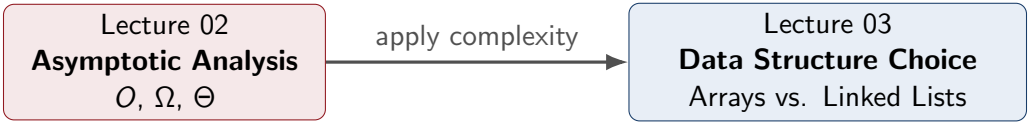
Presidency University

Lecture 03: Learning Objectives

By the end of this lecture, students should be able to:

1. Explain the **List ADT** and distinguish an ADT from its implementation.
2. Describe how **arrays/dynamic arrays** store and access elements.
3. Analyze array access, search, insertion, and deletion using asymptotic notation.
4. Explain why linked lists are useful when structural changes are frequent.
5. Trace and implement traversal, search, insertion, and deletion in a **singly linked list**.
6. Explain **doubly** and **circular** linked lists and when they are appropriate.
7. Compare array-based and linked representations by time, memory, locality, and use case.
8. Select a suitable list representation for a stated computational problem.

Connection with Lecture 02



Central Question

The same List ADT can be implemented in different ways. **Which representation gives the operations our problem needs most efficiently?**

What Is a Data Structure?

Definition

A **data structure** is a systematic way to organize, store, and manage data so that required operations can be performed efficiently.

Logical view

- What data is represented?
- What operations are allowed?
- What relationships exist among items?

Physical/implementation view

- How is memory organized?
- How are elements located?
- What must move when structure changes?

Examples

Arrays, linked lists, stacks, queues, trees, heaps, hash tables, and graphs are different structures because they organize relationships and operations differently.

Linear Data Structures

Linear Organization

In a linear data structure, elements form a sequence. Except at the ends, each element has a logical predecessor and successor.



Examples in the course: lists, arrays, linked lists, stacks, and queues.

Important

"Linear" describes the **logical order**; it does not require physically adjacent memory. Arrays are contiguous; linked-list nodes may be scattered.

Abstract Data Type (ADT) vs. Data Structure

ADT: What?

Defines the behavior of a collection through operations, without specifying storage details.

- values / elements
- valid operations
- expected behavior

Data Structure: How?

Provides a concrete representation and algorithms for those operations.

- array-based list
- linked list
- other implementations

List ADT $\Longrightarrow$ Array implementation or Linked implementation

The List ADT

A **list** is an ordered finite sequence:

$$L = \langle a_0, a_1, a_2, \ldots, a_{n-1} \rangle$$

where position matters and duplicate values may be allowed.

Typical query operations

- `size()` / `is_empty()`
- `get(i)`
- `find(x)`
- traversal / iteration

Typical update operations

- `set(i,x)`
- `insert(i,x)`
- `append(x)`
- `remove(i)` / remove value

Representation Independence

The List ADT says what these operations mean; the chosen representation determines their actual cost.

Array Representation: Core Idea

An array stores elements in **contiguous memory locations**.



If base address is B, element size is w bytes, and indexing starts at 0:

$$\text{address}(A[i]) = B + i \cdot w$$

Consequence

The address is computed directly from the index; therefore random access does not require scanning earlier elements.

Why Array Indexing Is $O(1)$

Suppose $A = [18, 7, 25, 11, 30]$ and we request $A[3]$.

1. The index 3 is known.
2. Compute address $B + 3w$.
3. Read the value stored there: 11.

The number of address-computation steps does not grow with n.

$$\boxed{T_{\text{access}}(n) = \Theta(1)}$$

Do Not Confuse

Access by index is $O(1)$, but **search by value** is generally $O(n)$ in an unsorted array because the desired index is unknown.

Array Traversal and Search

```
def linear_search(A, target):
    for i in range(len(A)):
        if A[i] == target:
            return i
    return -1
```

Complexity

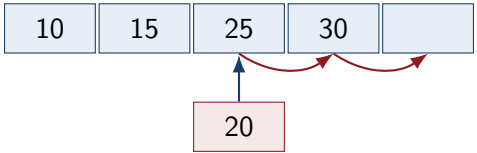
- Best case: $\Theta(1)$
- Worst case: $\Theta(n)$
- Average case: $\Theta(n)$
- Full traversal: $\Theta(n)$

Trace: target = 11

For $[18, 7, 25, 11, 30]$: compare with $18 \to 7 \to 25 \to 11$. Found at index 3 after four comparisons.

Insertion into an Array-Based List

Insert 20 at index 2 in $[10, 15, 25, 30]$.



Elements from the insertion position onward must shift right.

insert at end: amortized $O(1)$ but insert near front/middle: $O(n)$

Deletion from an Array-Based List

Delete the element at index 1 from $[10, 15, 20, 25, 30]$.



To keep the logical sequence compact, later elements shift left.

deletion by known index: $O(n)$ worst case

If deletion first requires searching for a value, search is also $O(n)$; the combined order remains $O(n)$.

Fixed Arrays vs. Dynamic Arrays

Fixed-size array

Capacity is decided when storage is allocated.

- simple layout
- predictable memory
- cannot grow beyond capacity

Dynamic array

Maintains a backing array with a **capacity** that may exceed current **size**.

- supports growth
- occasionally reallocates
- copies elements during resize

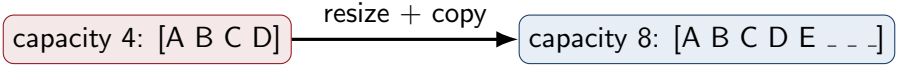
Python Connection

Python's built-in `list` behaves conceptually like a dynamic array: indexed access is fast, while insertions/deletions away from the end may shift references.

Dynamic Array Growth

When the backing array is full, a typical strategy is:

1. Allocate a larger array, often proportional to the old capacity.
2. Copy existing elements.
3. Insert the new element.
4. Release the old backing storage.



One resize costs $\Theta(n)$, but it does **not** happen on every append.

Why Append Can Be Amortized $O(1)$

Assume capacity doubles: $1, 2, 4, 8, \ldots$. The total number of copied elements before reaching size n is bounded by

$$1 + 2 + 4 + \cdots + \frac{n}{2} < n.$$

Together with n ordinary insertions, total work for n appends is $O(n)$.

$$\boxed{\frac{O(n)}{n} = O(1) \text{ amortized per append}}$$

Amortized $\neq$ Worst Case

A particular append that triggers resizing can be $O(n)$; the **average cost over a sequence of appends** is $O(1)$ under geometric growth.

Array-Based List: Complexity Summary

Operation	Typical / Amortized	Worst Case
Access by index	$O(1)$	$O(1)$
Update by index	$O(1)$	$O(1)$
Search unsorted	$O(n)$	$O(n)$
Append dynamic array	$O(1)$ amortized	$O(n)$
Insert at front/middle	$O(n)$	$O(n)$
Delete at front/middle	$O(n)$	$O(n)$
Traverse all elements	$O(n)$	$O(n)$

Strength

Excellent indexed access and cache locality.

The Problem That Motivates Linked Lists

Suppose a sequence changes frequently near the front or between known elements.

- Array insertion may require shifting many elements.
- Resizing may require allocating and copying a larger block.
- Contiguous storage must be available for the backing array.

Alternative Idea

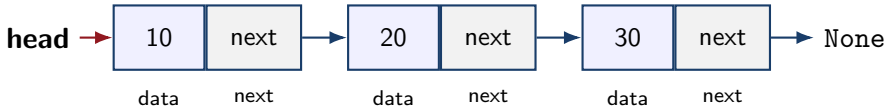
Store each element in a separate **node**. Instead of relying on physical adjacency, store a **reference/link** to the next node.

logical order through links, not through neighboring addresses

Singly Linked List: Node Structure

Each node in a singly linked list contains:

1. **data** — the value stored in the node;
2. **next** — a reference to the next node.



Key Invariant

Starting at `head` and repeatedly following the `next` reference visits the nodes in order:

$$10 \rightarrow 20 \rightarrow 30 \rightarrow \texttt{None}$$

The final node always has `next = None`.

A Minimal Node Class in Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.size = 0
```

Reference Model

In Python, `next` stores a reference to another object (or `None`). Conceptually, this plays the role of a pointer/link used to connect nodes.

An empty list is represented by:

`head = None,  size = 0`

Traversing a Singly Linked List

```
def display(self):
    current = self.head
    while current is not None:
        print(current.data, end=" -> ")
        current = current.next
    print("None")
```

Loop invariant: before each iteration, `current` refers to the next unprocessed node.

- Each iteration advances exactly one link.
- For n nodes, all n nodes are visited.

$$\boxed{T_{\text{traversal}}(n) = \Theta(n), \qquad \text{extra space} = \Theta(1)}$$

Searching a Singly Linked List

```
def find(self, target):
    current = self.head
    index = 0
    while current is not None:
        if current.data == target:
            return index
        current = current.next
        index += 1
    return -1
```

Best case: target at head $\Rightarrow O(1)$.

Worst case: target last/absent $\Rightarrow O(n)$.

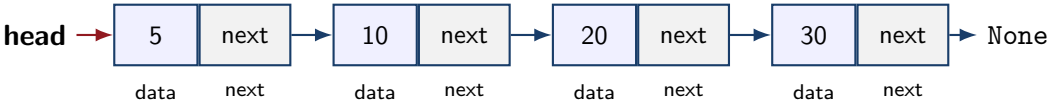
No Random Access

To reach position i, we must follow i links from the head. Thus indexed access is $O(i)$ and $O(n)$ in the worst case.

Insertion at the Beginning

Before: `head` $\rightarrow$ `10` $\rightarrow$ `20` $\rightarrow$ `30` $\rightarrow$ `None`

1. Create `new = Node(5)`.
2. Set `new.next = head`.
3. Set `head = new`.



Insertion at head $= \Theta(1)$

Why is it constant time?

We only change two references: `new.next` and `head`. No traversal of the list is required.

Why Pointer/Reference Update Order Matters

To insert `new` before the old head, use:

```
new.next = self.head
self.head = new
```

If we reverse the assignments:

```
self.head = new
new.next = self.head
```

then `new.next` points back to `new`; the original list is no longer reachable through `head`.

General Rule

Before overwriting a link that is your only route to existing nodes, **save/preserve the route first**.

Insertion at the End

Without a tail reference:

1. If empty, make the new node the head.
2. Otherwise traverse until `current.next is None`.
3. Set `current.next = new`.

$$\boxed{\Theta(n)}$$

With a maintained tail reference:

```
self.tail.next = new
self.tail = new
```

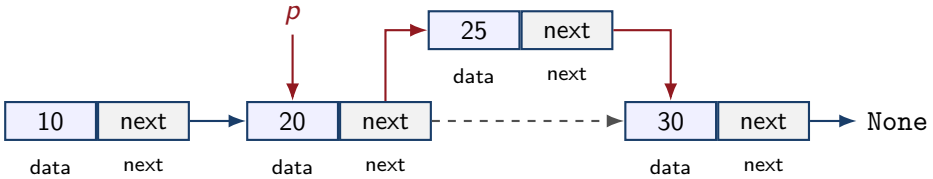
$$\boxed{\Theta(1)}$$

Design Insight

Extra metadata can improve operation cost, but every update must preserve that metadata correctly.

Insertion After a Known Node

Given a direct reference p to the node containing 20, insert 25 after it.



Correct order of updates:

```
new.next = p.next
p.next = new
```

Time Complexity

If p is already known, insertion after p takes

$\Theta(1)$

However, finding p from head may still require

$\Theta(n)$

Insertion at an Arbitrary Index

To insert at position i in a singly linked list:

1. Validate $0 \leq i \leq n$.
2. If $i = 0$, insert at head.
3. Otherwise traverse to node at $i - 1$.
4. Link the new node between predecessor and successor.

$$T(i) = \Theta(i) \quad \Rightarrow \quad T_{\text{worst}}(n) = \Theta(n)$$

Key Distinction

Linked lists make the **link modification** constant time once the location is known. They do not make locating an arbitrary index constant time.

Deleting the First Node

Before: head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ None

```
if self.head is not None:
    self.head = self.head.next
    self.size -= 1
```

After: head $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ None

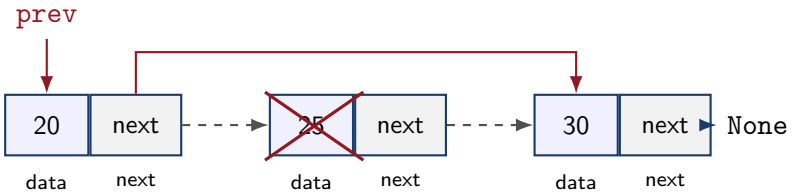
$$\boxed{\text{Delete head} = \Theta(1)}$$

Edge Case

Always handle the empty list. If a tail reference is maintained, deleting the only node must also set `tail = None`.

Deleting After a Known Predecessor

Suppose `prev` points to the node containing 20, and we want to remove 25.



Pointer update:

```
removed = prev.next
prev.next = removed.next
```

What happens?

`prev.next` is changed from the node containing 25 to the node containing 30. Therefore, node 25 is bypassed and removed from the list.

Deleting by Value

```
def remove_value(self, target):
    prev = None
    current = self.head
    while current is not None:
        if current.data == target:
            if prev is None:
                self.head = current.next
            else:
                prev.next = current.next
            self.size -= 1
            return True
        prev, current = current, current.next
    return False
```

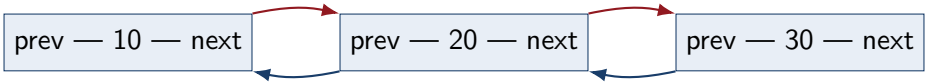
Worst-case time: $O(n)$ because the target may be last or absent.
Extra space: $O(1)$ for iterative traversal.

Singly Linked List: Complexity Summary

Operation	If position/node known	From head
Access by index	—	$O(n)$ worst case
Search by value	—	$O(n)$
Insert at head	$O(1)$	$O(1)$
Insert after known node	$O(1)$	search may make $O(n)$
Append, no tail	—	$O(n)$
Append, with tail	$O(1)$	$O(1)$
Delete head	$O(1)$	$O(1)$
Delete after known predecessor	$O(1)$	search may make $O(n)$
Traversal	—	$O(n)$

Doubly Linked List

Each node stores both `prev` and `next` references.



Advantages: backward traversal; deletion of a known node does not require searching for its predecessor.
Costs: extra reference per node; more links must be updated correctly.

Doubly Linked List: Inserting Between Two Nodes

Insert `x` between `a` and `b` where initially `a.next = b` and `b.prev = a`.

```
x.prev = a
x.next = b
a.next = x
b.prev = x
```

After insertion, the local invariant must hold:

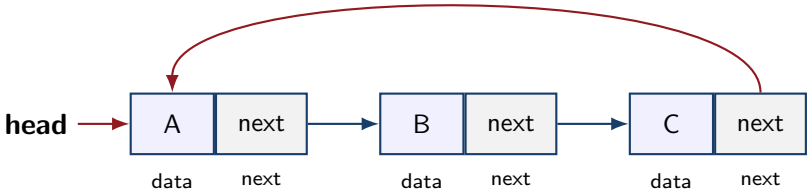
$$x.prev.next = x \quad \text{and} \quad x.next.prev = x$$

Common Bug

Updating only the forward links may make forward traversal appear correct while backward traversal is broken. Every structural change must preserve both directions.

Circular Linked Lists

In a circular linked list, the last node links back to the first instead of storing `None`.



Useful idea: after the last element, processing naturally continues from the first.
Typical applications: cyclic scheduling, round-robin processing, repeated playlists, and circular buffers.

Traversal Warning

Do not stop at `None`, because a circular linked list has no `None` at the end. Stop when traversal returns to the starting node (or after visiting a known number of nodes).

Array vs. Linked List: Structural Comparison

	Array / Dynamic Array	**Linked List**
Memory layout	contiguous backing storage	nodes may be non-contiguous
Indexed access	direct, $O(1)$	sequential, $O(n)$ worst case
Insert/delete middle	shifting, $O(n)$	$O(1)$ relink if node/predecessor known
Search unsorted	$O(n)$	$O(n)$
Per-element overhead	low	link reference(s) per node
Cache locality	usually strong	usually weaker
Capacity growth	may resize/copy	node-by-node allocation

The Hidden Cost: Memory Locality

Big-O does not capture every performance factor.

Array

Adjacent elements are stored together, so sequential traversal often benefits from spatial locality and cache behavior.

Linked List

Nodes may be scattered; following references can require accessing unrelated memory locations, and each node stores link metadata.

Lesson

Two operations can both be $O(n)$ yet have noticeably different real execution times because asymptotic complexity ignores constants, cache effects, allocation overhead, and language/runtime details.

Choosing the Representation

Requirement	Usually Favor
Frequent random/indexed reads	Dynamic array
Compact storage and fast iteration	Dynamic array
Frequent insertion/removal at front	Linked list
Frequent insertion/removal after a known node	Linked list
Need backward movement through nodes	Doubly linked list
Repeated cyclic traversal	Circular linked list

Decision Principle

Choose according to the **operation mix and constraints**, not according to which structure sounds more advanced.

Worked Problem 1: Which Structure?

A program stores 100,000 readings. It repeatedly reads values by index and rarely inserts elements except at the end.

1. Dominant operation: indexed access.
2. Array access: $O(1)$.
3. Linked-list access: $O(n)$ worst case.
4. End insertion in a dynamic array: amortized $O(1)$.

Choice

Dynamic array because it optimizes the dominant operation and still supports efficient append behavior.

Worked Problem 2: Which Structure?

An editor maintains a current node in a sequence and frequently inserts or removes items immediately around that known location; backward movement is also required.

1. Random indexing is not the main requirement.
2. Insert/delete around a known node should avoid shifting.
3. Backward navigation is explicitly required.

Choice

Doubly linked list because local relinking is constant time once the node is known, and `prev` supports backward traversal.

Worked Trace: Insert into a Singly Linked List

Initial list:

$$10 \to 20 \to 40 \to \texttt{None}$$

Insert 30 after 20. Let `p` point to node 20.

1. Create node 30.
2. `new.next = p.next` $\Rightarrow$ new points to 40.
3. `p.next = new` $\Rightarrow$ 20 points to 30.

Final list:

$$\boxed{10 \to 20 \to 30 \to 40 \to \texttt{None}}$$

Why step 2 first? It preserves the only link from 20 to the remainder of the list before that link is overwritten.

Common Mistakes and How to Avoid Them

- **Saying linked-list insertion is always $O(1)$:** location/search may cost $O(n)$.
- **Calling Python list a linked list:** it behaves as a dynamic array.
- **Losing nodes during insertion:** preserve the old successor before overwriting a link.
- **Ignoring empty/single-node cases:** head/tail updates differ at boundaries.
- **Off-by-one errors:** clearly define valid indices for access $(0 \ldots n-1)$ and insertion $(0 \ldots n)$.
- **Infinite circular traversal:** use a return-to-start stopping condition.
- **Comparing only Big-O:** also consider memory overhead and locality.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array. $O(1)$
2. Search an unsorted array for value x.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array. $O(1)$
2. Search an unsorted array for value x. $O(n)$
3. Read position i in a singly linked list from head.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array. $O(1)$
2. Search an unsorted array for value x. $O(n)$
3. Read position i in a singly linked list from head. $O(n)$
4. Insert at the head of a singly linked list.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array. $O(1)$
2. Search an unsorted array for value x. $O(n)$
3. Read position i in a singly linked list from head. $O(n)$
4. Insert at the head of a singly linked list. $O(1)$
5. Insert after a node when a direct reference to that node is already given.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array. $O(1)$
2. Search an unsorted array for value x. $O(n)$
3. Read position i in a singly linked list from head. $O(n)$
4. Insert at the head of a singly linked list. $O(1)$
5. Insert after a node when a direct reference to that node is already given. $O(1)$
6. Delete a value from a singly linked list when its location is unknown.

Quick Check: Predict the Complexity

For a list of n elements, state the worst-case complexity.

1. Read `A[i]` from an array. $O(1)$
2. Search an unsorted array for value x. $O(n)$
3. Read position i in a singly linked list from head. $O(n)$
4. Insert at the head of a singly linked list. $O(1)$
5. Insert after a node when a direct reference to that node is already given. $O(1)$
6. Delete a value from a singly linked list when its location is unknown. $O(n)$

In-Class Coding Exercise

Implement a `SinglyLinkedList` supporting:

1. `append(x)`
2. `prepend(x)`
3. `find(x)`
4. `insert_after(target, x)`
5. `remove(x)`
6. `display()`

Test Cases

Test: empty list; one-node list; insertion at beginning/end; duplicate values; missing target; deleting the head; deleting the last node.

Analysis Requirement

For each method, state worst-case time complexity and explain which part of the algorithm determines

Practice Problems

1. Starting with `head -> 8 -> 13 -> 21 -> None`, draw every reference update needed to insert 17 after 13.
2. Show the reference update required to delete 13 from the resulting list.
3. Explain why `A[50000]` is $O(1)$ for an array even though the index is large.
4. A program performs 95% indexed reads and 5% insertions. Which representation would you initially choose, and why?
5. A singly linked list stores only `head`. What is the complexity of appending one element? How can an additional `tail` reference change it?
6. Explain one situation where a doubly linked list is preferable despite its additional memory overhead.

Lecture 03 Summary

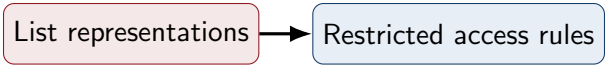
- A **List ADT** defines ordered-sequence behavior independently of implementation.
- Arrays provide $O(1)$ **indexed access** through contiguous storage and address calculation.
- Dynamic arrays grow by occasional reallocation; append is typically **amortized** $O(1)$.
- Linked lists represent order through references and support efficient local relinking.
- Singly linked lists require sequential access; doubly linked lists add backward links; circular lists connect the end back to the beginning.
- The correct representation depends on **dominant operations, memory costs, locality, and boundary cases**.

Core Takeaway

Data structure choice is an algorithmic design decision.

From General Lists to Restricted Linear ADTs

Stacks and Queues



Prepare: review insertion/deletion at the ends of arrays and linked lists.

Reference and Reading Guidance

Recommended Course Text

Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser, *Data Structures and Algorithms in Python*, Wiley.

Focus your reading on:

- sequence/list abstractions;
- array-based sequences and dynamic arrays;
- linked structures and positional/reference-based updates;
- operation-cost comparison using asymptotic analysis.

Study Method

Do not memorize the complexity table alone. For every complexity, be able to explain **what work is performed as n grows**.